Group #8 Assignment Report

An Evolutionary Algorithm for the Travelling Salesman Problem

Filip Sjögren Jesper Claesson Joakim Möll
Murillo Moraes Giroldo Sebastian Andersson

October 12, 2025

1 Introduction

This assignment, part of the course TARI29 – Artificial Intelligence, focuses on the study and practical implementation of algorithms inspired by evolutionary computation. The main objective is to design and evaluate a Genetic Algorithm (GA) capable of solving a combinatorial optimization problem. For this purpose, the well-known Travelling Salesman Problem (TSP) was selected as the test case due to its computational complexity and relevance in heuristic optimization research.

The work involved implementing the core components of a GA, including population representation, fitness evaluation, selection, crossover, mutation and analyzing how these evolutionary operators contribute to the algorithm's ability to approximate optimal solutions. The algorithm was developed from scratch in Python, following an imperative programming paradigm, in accordance with the course requirements.

The study aimed to evaluate the efficiency and robustness of the implemented GA across different experimental conditions. Although several parameters such as population size, mutation rate, and number of generations were explored, the focus of the project remained on understanding the underlying evolutionary process and assessing the algorithm's capacity to adaptively search for high-quality solutions.

By combining algorithmic design with empirical evaluation, this assignment provided a hands-on understanding of how evolutionary principles can be employed to tackle complex optimization problems where traditional exact methods become computationally infeasible.

2 Travelling Salesman Problem (TSP)

Problem. Given a complete weighted graph $G = (V, E)$ with $|V| = n$ cities and distances $d_{ij} \geq 0$, the TSP seeks a minimum-length Hamiltonian cycle visiting each city exactly once and returning to the start. We encode a tour as a permutation π and evaluate it by

$$f(\pi) = \sum_{k=1}^{n} d_{\pi_k, \pi_{(k \bmod n)+1}},$$

which matches our implementation (closing the tour at the end). The combinatorial search space ($n!$) motivates heuristic methods.

Related evolutionary work (anchors). In permutation-encoded TSP, problem-aware crossovers that preserve relative order are standard baselines; a common pattern is to copy a contiguous segment from one parent and fill remaining positions by the second parent's order, ensuring a valid permutation without repair [1]. Tournament selection provides simple, tunable selection pressure controlled by the tournament size k [2]. Classic mutation operators for permutations include *swap*, *insertion*, and *inversion*; low mutation rates are recommended to preserve search focus [3].

Motivation for our design. Our goal is a clear, from-scratch GA aligned with these guidelines: path (permutation) representation with tour-length fitness; tournament selection (tunable pressure); an order-preserving two-cut crossover that yields valid tours without repair; and a lightweight *swap* mutation for diversity [1, 2, 3].

3 Genetic Algorithm

3.1 Evolutionary Approach

The algorithm begins by initializing a population of *num_paths* randomly generated solutions, which forms the starting point for the search. Through *num_generations* iterations, the population evolves into optimized combinations of paths, gradually guiding the population towards solutions of higher fitness.

In order to achieve balance between exploring new solutions while preserving and exploiting optimal candidates, the evolutionary algorithm proceeds through four key stages:

- *Tournament selection* - parents are chosen based on their fitness from the starting city, where the shorter routes have a better fitness
- *Crossover* - two parents are combined to produce a new offspring path. A random segment from one parent is copied, and the remaining cities are filled in the same order as they appear in the second parent
- *Mutation* - a small random change is applied to the route, by swapping two cities. This helps prevent the algorithm from getting trapped in a local minima
- *Elitism* - a few of the best routes are directly copied to the next generation to ensure that the best solutions are preserved

This process continues until termination, and the latest solution for the algorithm will be the shortest path calculated.

3.1.1 Tournament selection

Tournament selection was chosen for its efficiency and robustness when fitness values vary a lot across the population. This method picks k individuals at random, and the winner is determined by the candidate with the best fitness (shortest route). This route wins the tournament and is chosen as a parent. The process is repeated to obtain two winners for a crossover. The parameter k controls the selection pressure – a larger k will increase the likelihood of selecting the fittest individuals, while a smaller k promotes more diversity, increasing the likelihood of varied fitness levels in participants.

To validate this choice, we compared the algorithm's performance over 100 runs with tournament selection ($k = 5$) and 100 runs without it, using random parent selection. The version with tournament selection achieved a much lower mean best distance (**9,123**) compared to the random-parent version (**15,724**), while the known optimum for the Berlin52 problem is **7,542**. These results confirm that the tournament approach provides a better balance between selective pressure and population diversity, justifying its use in all subsequent experiments.

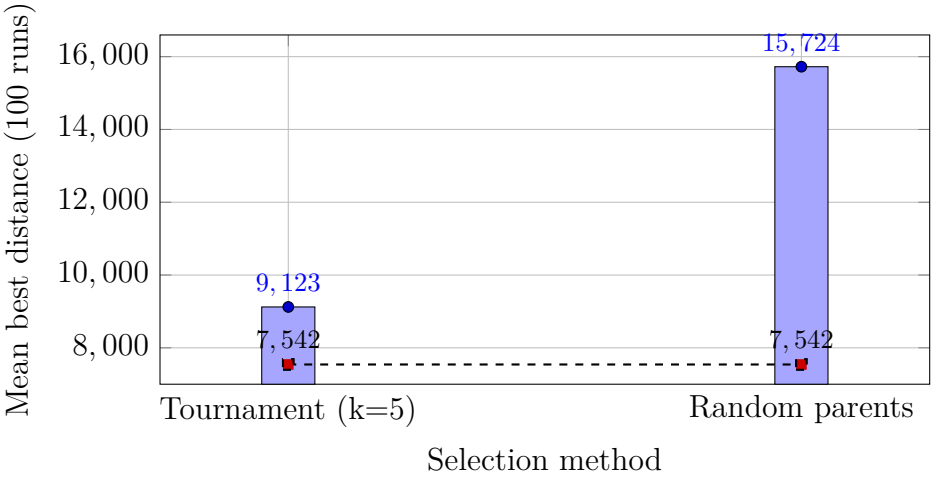

Figure 1: Comparison between tournament selection ($k = 5$) and random parent selection over 100 runs. The dashed line marks the Berlin52 optimum (7,542).

3.1.2 Crossover

The crossover operator is based on a segment method where a continous part of one parent's route is copied directly into the child, the remaining cities are filled from the second parent, in the same **relative order**. This approach ensures that the child represents a valid TSP route where each city only appears once. Note that city 0 remains fixed at the beginning of the route to maintain a valid trip.

Visualization of order crossover with size $n = 7$:

Parent 1:	$[0, 3, 1, 2, 5, 4, 6]$
Parent 2:	$[0, 2, 1, 5, 4, 3, 6]$
Segment from parent 1:	$[0, 3, 1, \mathbf{2,5,4}, 6]$
Fill with remaining:	$[0, \mathbf{1}, \mathbf{3}, 2, 5, 4, \mathbf{6}]$

3.1.3 Mutation

The mutation operator introduces small random changes in the routes to maintain diversity within the population and prevent the algorithm from converging too early to a suboptiomal solution. In this case we used a simple but effective *swap mutation*. This method selects two cities by random and swaps their positions with a probability p_m. This makes the algorithm explore new areas of the search space without interfering the overall structure of the route. This simple method gives

the algorithm just enough variation to help the population escape a local minima point.

3.1.4 Elitism

The elitist approach is included to ensure that the best solution found so far is not lost in the next generation. At the end of each generation, *num_best* top-preforming routes, called *elites*, are copied directly into the new population without any modification. This guarantees that the overall quality of the population never decreases from one generation to another. The use of elitism helps the algorithm converge **faster** towards better solutions, strong individuals are always preserved and allowed to influence next generations.

3.2 The Final Algorithm: Pseudo Code

The following pseudo code presents the complete version of the proposed algorithm, combining the critical stages described in the previous sections.

Algorithm 1 Genetic Algorithm

```
paths = init_popuation(size, num_paths)

i ← 0
while i < num_generations do

    best_paths = select_best_paths(paths, matrix, num_best)
    new_paths ← copy(best_paths)

    while |new_paths| < num_paths do

        parent_1 ← tournament(paths, matrix, k_tournaments)
        parent_2 ← tournament(paths, matrix, k_tournaments)

        attempts ← 0
        while parent_2 is parent_1 and attempts < 3 do

            parent_2 ← tournament_select(paths, matrix, k_tournaments)
            attempts ← attempts + 1

        end while

        child ← crossover(parent_1, parent_2)
        child ← mutate(child, mutation_rate)
        append(new_paths, child)

    end while

    paths ← new_paths
    i ← i + 1

end while
```

3.3 Solution representation

3.3.1 Candidates

The candidate solutions are represented as an ordered sequence of city indices, where two consecutive indices correspond to a distance defined in an adjacency

matrix of pairwise distances. For example, candidate $[0, 3, 1, 2, 5, 4, 6]$ represents a possible tour visiting cities in that order, and $matrix[0][3]$ represents the distance between city 0 and city 3.

As the optimal solution to the TSP presume a visit to every city *once*, the representation of each candidate is a permutation of indices ranging from 0 to $cities - 1$. Each population consists of *num_path* candidate solutions.

The genetic operations crossover and mutation are applied directly on the candidates, while the tournament selection evaluates the candidates based on their total tour distance, derived from the adjacency matrix. To preserve only valid solutions using the same starting point, the city indexed 0 is never modified. Therefore, no crossover or mutation is performed on this index.

3.3.2 Output

All the individuals in the population represents a complete route visiting all cities exactly one time. This makes the representation implemented as a permutation of city indices where each gene corresponds to a city. City 0 is fixed as both start and ending point which ensures that every solution presented forms a valid trip around all cities.

The final solution is presented like this :

- **Best path:** $[p_0, p_1, p_2, \ldots, p_n, p_0]$
- **Best distance:** $F_{\min}$
- **Optimal distance (reference):** F^*
- **Number of cities visited:** n

3.4 Fitness function

The quality of each route is determined by using a fitness function that measures its total distance traveled. The objective of the TSP is to minimize the total distance of the trip, meaning that shorter routes are considered better. During selection and elitism, individuals with lower total distances are preferred since they represent more optimal solutions. The total distance is calculated by summing the distances between consecutive cities in the route, including the return to the starting city. This approach keeps the evaluation simple and ensures that every route can easily be compared based on how efficiently it connects all of the population.

$$F_{min} = \sum_{i=0}^{n-1} d(p_i, p_{(i+1)}) \tag{1}$$

where p_i represents the i-th city in the route and $d(p_i, p_j)$ is the distance between city i and city j. The goal of the algorithm is to minimize F_{min}, i.e., to find the route P^* that yields the shortest possible total distance.

4 Experimental part

4.1 Hardware and Software

All experiments were executed locally by group members on heterogeneous personal machines (CPU/RAM/GPU vary across participants). To ensure software consistency, every run used Python 3.12.1 from within VS Code (stable channel) with the same codebase and configuration files. The implementation relies on the Python standard library plus NumPy and Pandas for data handling; plots were produced with PGFPlots.

Because hardware differs across contributors, all quality metrics (best, median, std of tour length) are hardware-agnostic. Wall-clock time should be interpreted as indicative only and not used for fine-grained cross-machine comparisons. When reporting compute effort, we therefore also reference a hardware-independent proxy, the number of fitness evaluations:

$$\text{evals} \;=\; (\texttt{num_paths}) \times (\texttt{num_generations}) \quad \text{(per run, ignoring elitism).}$$

Unless otherwise stated, each configuration was run with multiple independent random seeds to capture stochastic variability.

4.2 Datasets

We used symmetric, Euclidean TSPLIB instances: `ulysses16`, `bays29`, `berlin52`, and `eil76`. Distances are computed from coordinates using standard Euclidean metric without rounding. For each instance we report the best-known optimum F and the percentage gap of our best and median solutions: $\text{gap}(\%) = 100 \times (F_{\text{GA}} - F^{)/F}$.

4.3 Parameter Tuning and Sensitivity Analysis

Before evaluating the final performance of the Genetic Algorithm (GA), an additional experiment was conducted to calibrate its main hyperparameters. This tuning stage aimed to determine a configuration that offered a good balance between solution quality, runtime, and consistency across multiple executions. Given the stochastic nature of evolutionary algorithms, this calibration was crucial to ensure that the chosen parameter set produced reliable and reproducible results.

Experimental setup. Five key parameters were varied systematically:

- **Mutation rate:** 1%, 3%, 5%, 8%, and 10%
- **Population size:** 100, 250, and 400 individuals
- **Number of generations:** 500, 1000, and 2000
- **Number of elites (*num_best*):** 2 and 4
- **Tournament size (*k_tournament*):** 3, 5, and 7

All possible combinations of these parameters were tested, resulting in a total of 270 unique configurations. Each configuration was executed ten times using different random seeds to account for stochastic variability, leading to a total of 2,700 independent runs. For each run, the algorithm recorded the best distance obtained and the corresponding runtime. After execution, results were aggregated into summary statistics including the mean, median, and standard deviation of the best distances per configuration. The analysis focused not only on minimizing the mean distance, but also on identifying stable configurations with low variability and reasonable computational cost.

Results. Figure 2 illustrates the influence of the mutation rate on the mean best distance, with all other parameters fixed. The lowest mean distance was obtained with a mutation rate of 1%, indicating that a small degree of random variation is sufficient to preserve population diversity without disrupting convergence. Higher mutation rates (above 5%) consistently degraded performance, likely due to excessive randomization in the population.

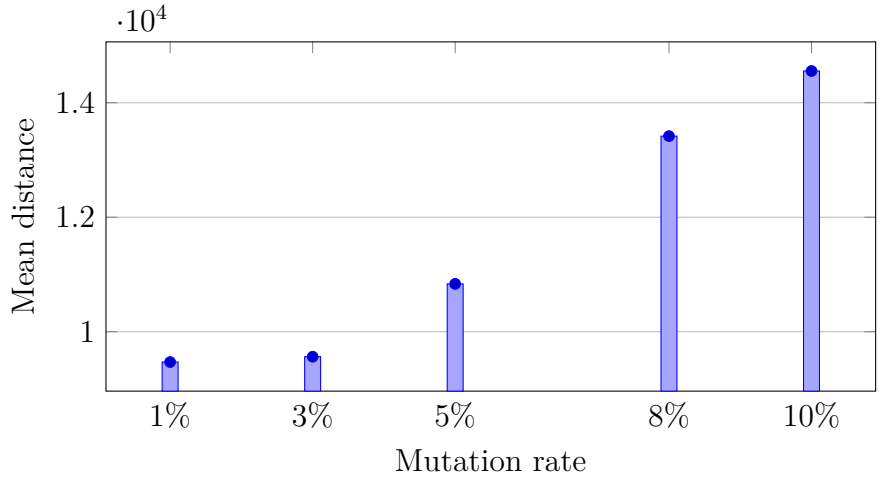

Figure 2: Effect of mutation rate on the mean best distance (error bars show one std).

Figure 3 presents the effect of population size on the mean best distance, with error bars representing one standard deviation. Larger populations led to better results and smaller variance, confirming that maintaining genetic diversity enhances exploration and prevents premature convergence. Among the tested values, a population size of 400 individuals produced the best average distance with the lowest variability.

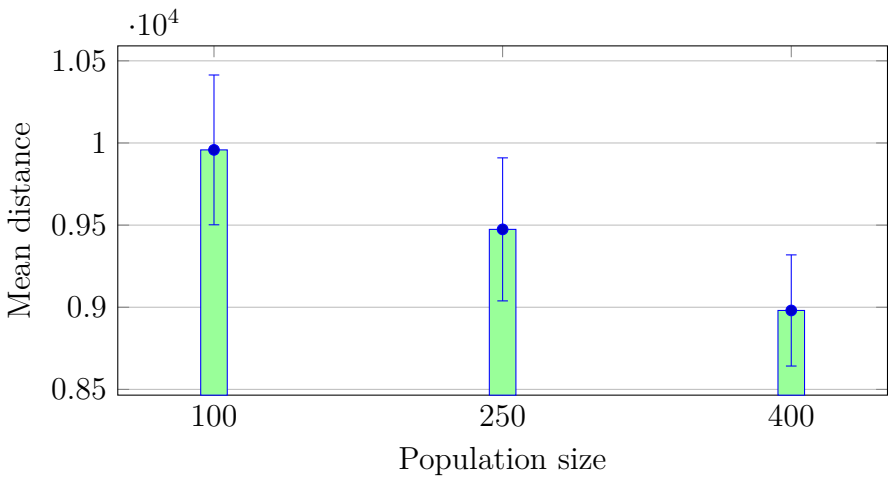

Figure 3: Effect of population size on the mean best distance (error bars show one standard deviation). A population of 400 individuals achieved the best performance and the lowest variability.

Figure 4 explores the trade-off between runtime and solution quality. The Y-axis represents the mean best distance, while the X-axis shows the mean runtime in seconds. Data points are color-coded by population size (red = 100, blue = 250, green = 400), and different markers denote the number of generations (diamonds = 500, squares = 1000, triangles = 2000). The plot reveals two clear trends: increasing population size consistently improves the quality of solutions, while increasing the number of generations beyond 1000 yields diminishing returns. Specifically, for a population of 400, the mean best distance improved marginally from approximately 8950 at 1000 generations to around 8800 at 2000 generations, whereas the runtime more than doubled. Thus, the additional computational cost was not justified by the small performance gain.

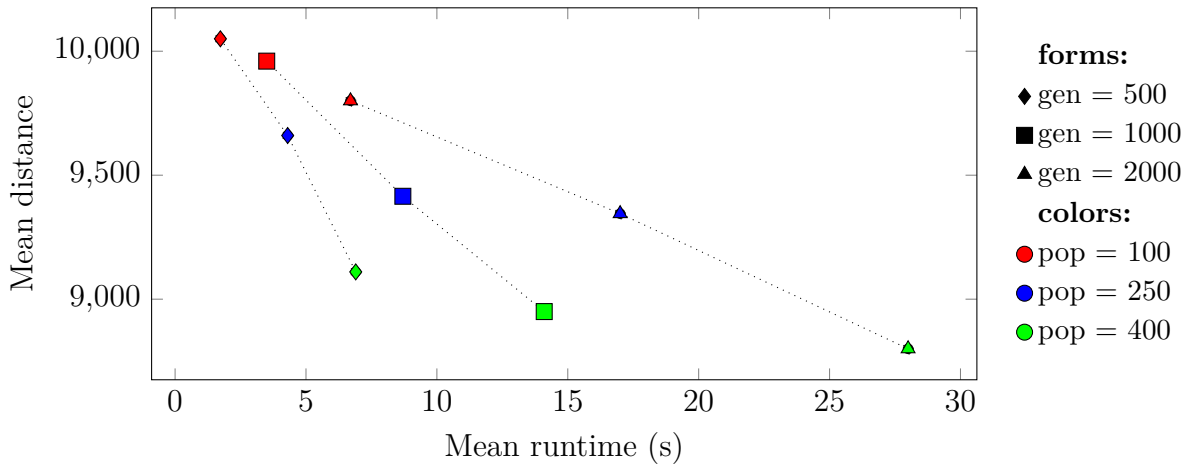

Figure 4: Trade-off between mean runtime and solution quality. Colors represent population size (red = 100, blue = 250, green = 400), and markers denote number of generations (diamonds = 500, squares = 1000, triangles = 2000). The results indicate diminishing returns beyond 1000 generations.

Selected configuration. Based on these experiments, the configuration

$$\begin{aligned} \texttt{mutation_rate} &= 0.01, \\ \texttt{num_paths} &= 400, \\ \texttt{num_generations} &= 1000, \\ \texttt{num_best} &= 4, \\ \texttt{k_tournament} &= 5 \end{aligned}$$

was selected as the final setup.

This parameter set achieves a strong compromise between accuracy, stability, and computational efficiency. It was subsequently used in all remaining experiments to evaluate the final performance of the Genetic Algorithm.

5 Results and Analysis

This section presents the experimental results obtained from the developed Genetic Algorithm and analyzes its performance in solving the Travelling Salesman Problem. The analysis includes a comparison with a baseline algorithm, an evaluation of consistency across multiple runs, and an examination of the algorithm's convergence behavior over generations.

5.1 Baseline Comparison with Prim's Algorithm

To provide a reference point for evaluating the Genetic Algorithm, Prim's algorithm is introduced to construct a Minimum Spanning Tree over the set of cities, from which a tour was created by traversing the MST. Although this approach does not guarantee optimality, it offers a computationally efficient solution that can be used for comparison of the GA's performance.

Table 1 presents the results obtained for several different TSP instances. Both the GA and Prim's algorithm were executed under identical conditions, and as shown in the table, the GA consistently outperformed the baseline across all test cases, achieving tour lengths much closer to the known optima.

Table 1: Comparison of Genetic Algorihm performance against Prim's algorithm and known optima.

Instance	**Optimum**	**GA**	**Baseline**	**time (s)**
ulysses16	6859	6972	7788	6.97
bays29	2020	2164	2677	10.79
berlin52	7452	8626	10402	20.96
eil76	538	630	731	30.52

Besides confirming the superiority of the Genetic Algorithm over Prim's algorithm, the results in Table 1 reveal additional patterns. As the problem size increased, the runtime grew approximately linearly, while the relative deviation from the optima increased mildly. This indicates that the GA scales efficiently across multiple TSP instances and maintains strong performance even as the search space expands.

5.2 Evaluation Of Consistency

To further evaluate the accuracy and consistency of the developed Genetic Algorithm, we analyzed its performance across 25 independent runs for each TSP instance. For each instance, the median best tour length was computed and compared with the known optimal solution. Figure 5 illustrates this comparison, showing how closely the GA approximates the optimal tour while maintaining stability across repeated executions.

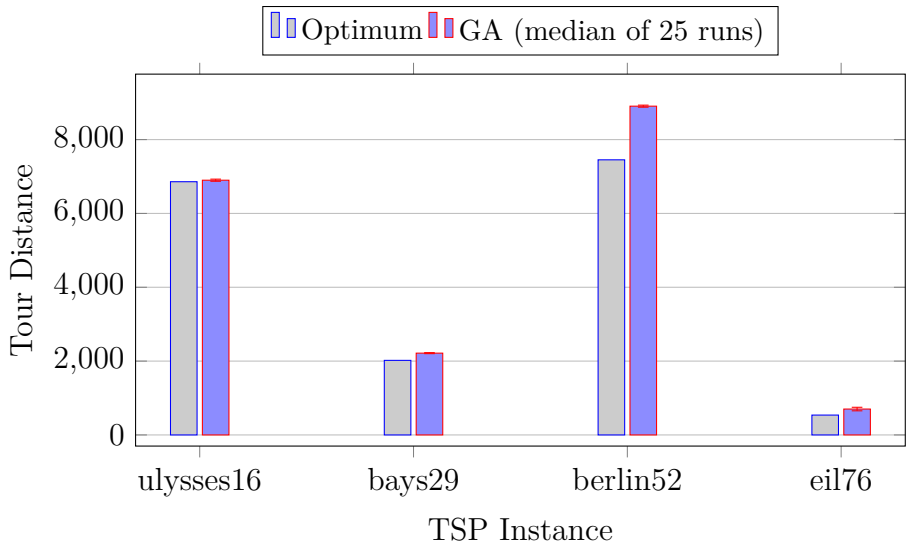

Figure 5: median

As shown in Figure 5, the Genetic Algorithm produced tour lengths consistently close to the known optima for all tested instances. The gap between the GA results and the optimal solutions remained small, indicating that the algorithm reliably converges toward high-quality solutions even under stochastic variation. Larger instances, such as *berlin52* and *eil76*, showed slightly higher deviations, which is expected given their larger search spaces and greater combinatorial complexity. Overall, these results demonstrate that the GA scales well with problem size while preserving solution quality.

5.3 Convergence Behavior

To better understand the convergence behavior of the GA, we tracked how the tour length successively evolved over generations. Figure 6 illustrates three independent

runs of the GA on the same TSP instance. The goal of this analysis is to observe whether the algorithm consistently converges toward high-quality solutions and how the rate of improvement changes over time.

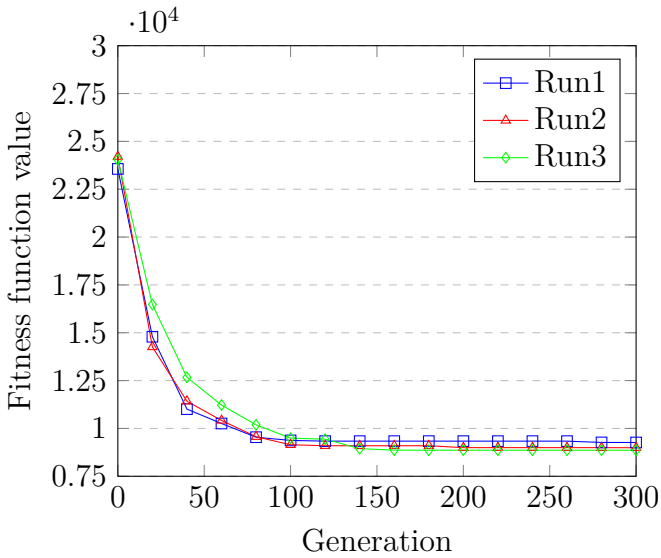

Figure 6: Example of convergence analysis.

As shown in Figure 6, all three runs displays a rapid improvement in solution quality during early generations, followed by a stabilization phase after approximately 100-150 generations. This behavior indicates that the GA quickly explores the search space and converges toward high-quality solutions within a relatively small amount of iterations. Overall, this shows a good balance between in the parameter setup.

6 Conclusions

In this study, a Genetic Algorithm (GA) was designed and implemented to evaluate its performance in optimizing the Travelling Salesman Problem (TSP). The main goal was to understand how evolutionary principles such as selection, crossover, mutation, and elitism can be used to find near-optimal solutions to complex optimization problems.

Through several experiments and parameter tuning, the algorithm demonstrated strong performance across multiple TSP instances, consistently producing high-quality routes with reasonable computational cost. As shown in Table 1, the GA

compared to the baseline Prim's algorithm consistently generated shorter tour lengths while maintaining reasonable runtimes. The final configuration also achieved stable results over multiple runs, as illustrated in Figure 5, showing the algorithm's reliability and robustness.

If this study were to be continued, some possible directions for improvement include implementing dynamic population sizes depending on the problem size to improve efficiency, exploring more problem-specific crossover functions and tournament selection (such as roulette), and applying adaptive mutation rates.

References

[1] A. Hussain, Y. S. Muhammad, M. N. Sajid, I. Hussain, A. M. Shoukry, and S. Gani, "Genetic algorithm for traveling salesman problem with modified cycle crossover operator," *Computational Intelligence and Neuroscience*, vol. 2017, pp. 1–7, 2017.

[2] B. L. Miller and D. E. Goldberg, "Genetic algorithms, tournament selection, and the effects of noise," *Complex Systems*, vol. 9, pp. 193–212, 1995.

[3] N. Kumar, Karambir, R. K., *et al.*, "A study of genetic algorithm to solve travelling salesman problem," *Journal of Global Research in Computer Science*, vol. 3, no. 3, pp. 33–37, 2012.